

**Gestión y clasificación de líneas de producción industriales mediante operaciones de  
conjuntos y lógica proposicional en C++**

Julio Aaron Andrade Chicaiza

Edwin David Guachi Cabrera

Jaime Mikael Hidalgo Nogales

Erick David Sabando Castillo

Anthony Alexander Vilaña Quishpe

Asesor

Washington Eduardo Loza Herrera

Universidad de las Fuerzas Armadas ESPE

Departamento de Ciencias de la Computación

Periodo

S-I ABR 26 - AGO 26

## Resumen

Este trabajo describe el desarrollo de un sistema para el análisis y control de líneas de producción industrial, basado en la combinación de un enfoque práctico y uno teórico. En la parte aplicada, el sistema permite registrar líneas, clasificar su estado operativo mediante lógica proposicional y agruparlas usando operaciones de conjuntos. En el plano teórico, se analizan los doce estados posibles derivados de tres variables clave, se calculan probabilidades discretas y se modela el coste computacional a través de una ecuación de recurrencia de crecimiento lineal. Para establecer una jerarquía de eficiencia entre estos estados, se diseñó un diagrama de Hasse. La implementación se realizó en C++ evitando el uso de bibliotecas de contenedores estándar para mantener un control directo sobre la memoria. Los resultados confirman que la clasificación es precisa y que el análisis matemático proporciona una estructura sólida para la toma de decisiones en plantas industriales.

**Palabras clave:** producción, lógica, conjuntos, probabilidad, jerarquía.

### **Abstract**

This paper describes the development of a system for the analysis and control of industrial production lines, based on a combination of practical and theoretical approaches. In the applied part, the system allows for the registration of lines, the classification of their operational state using propositional logic, and the grouping of lines using set operations. In the theoretical part, the twelve possible states derived from three key variables are analyzed, discrete probabilities are calculated, and the computational cost is modeled using a linear growth recurrence equation. A Hasse diagram was designed to establish an efficiency hierarchy among these states. The implementation was carried out in C++, avoiding the use of standard container libraries to maintain direct control over memory. The results confirm that the classification is accurate and that the mathematical analysis provides a robust framework for decision-making in industrial plants.

***Keywords:*** production, logic, sets, probability, hierarchy.

|                                                                |   |
|----------------------------------------------------------------|---|
| Introducción .....                                             | 7 |
| Objetivos .....                                                | 7 |
| Objetivo General.....                                          | 7 |
| Objetivos Específicos .....                                    | 7 |
| Marco Teorico.....                                             | 7 |
| Principios de Conteo y Combinaciones .....                     | 7 |
| Regla del Producto y De La Suma.....                           | 7 |
| Combinaciones .....                                            | 7 |
| Diseño y Arquitectura del Sistema .....                        | 7 |
| Definición de Estados de Producción (Principos de conteo)..... | 7 |
| Identificación de variables clave.....                         | 7 |
| Espacio muestral .....                                         | 7 |
| Análisis de Escenarios (Combinaciones) .....                   | 7 |
| Selección de casos .....                                       | 7 |
| Aplicación de fórmulas combinatorias .....                     | 7 |
| Modelo de Probabilidad Discreta .....                          | 7 |
| Definición del espacio muestral.....                           | 7 |
| Cálculo de las probabilidades .....                            | 7 |
| Ecuaciones de Recurrencia.....                                 | 7 |
| Identificar Operaciones Criticas .....                         | 7 |
| Planteamiento de la Ecuación de Recurrencia .....              | 7 |
| Análisis del crecimiento .....                                 | 7 |
| Diagrama de Hasse y Analisis de Complejidad.....               | 7 |

|                                  |   |
|----------------------------------|---|
| Pruebas .....                    | 7 |
| Prueba de Conteo .....           | 7 |
| Prueba de Probabilidad .....     | 7 |
| Prueba de Recurrencia .....      | 7 |
| Prueba de Jerarquia .....        | 7 |
| Complejidad .....                | 7 |
| Figuras .....                    | 7 |
| Tablas .....                     | 7 |
| Conclusiones .....               | 7 |
| Referencias Bibliográficas ..... | 7 |

### **Tabla de contenido**

|                                        |    |
|----------------------------------------|----|
| 1Diagrama de Hasse del Algoritmo ..... | 21 |
|----------------------------------------|----|

### **Lista de Figuras**

|                                                                                            |    |
|--------------------------------------------------------------------------------------------|----|
| <b>Tabla 1</b> <i>Matriz Para Representar El Grafo De La Relacion De Las Líneas.</i> ..... | 23 |
| Tabla II <i>Matriz Para Representar El Grafo de la Relacion de las Líneas</i> .....        | 23 |
| Tabla III <i>Significado de Cada Estado de la Matriz de la Relacion</i> .....              | 23 |
| Tabla IV .....                                                                             | 24 |

### **Lista de Tablas**

|                                                                    |    |
|--------------------------------------------------------------------|----|
| Tabla IV <i>Complejidad de las Funciones en el Algoritmo</i> ..... | 24 |
| <b>Apéndice A</b> <i>Ejemplo de Apéndice</i> .....                 | 21 |

## Introducción

Las plantas industriales actuales requieren sistemas de supervisión constantes que no solo registren datos, sino que analicen y clasifiquen el rendimiento de manera inteligente y escalable. El presente proyecto aborda esta necesidad mediante el diseño e implementación de un Sistema de Análisis Lógico para el Control de Producción Industrial, desarrollado en C++, enfocado en la empresa TechFactory.

El proyecto se estructura en dos fases complementarias. En una primera instancia, el sistema se fundamenta en la lógica proposicional y la teoría de conjuntos, utilizando estructuras de datos dinámicas (listas enlazadas, pilas y colas) para evaluar en tiempo real el estado de las líneas de producción (operativas, en bajo rendimiento o detenidas) y gestionar sus incidencias mediante un motor de inferencia lógico. En una segunda fase, el sistema evoluciona hacia un enfoque matemático analítico y predictivo. A través de la aplicación de principios de conteo, combinatoria y probabilidad discreta, se modelan de forma exhaustiva los escenarios teóricos de producción. Además, se introduce la teoría de conjuntos parcialmente ordenados para jerarquizar los estados de eficiencia mediante un Diagrama de Hasse, culminando con un riguroso análisis de complejidad computacional y ecuaciones de recurrencia. Esto garantiza que el sistema no solo resuelva el problema lógico actual, sino que ofrezca un rendimiento predecible y óptimo frente al crecimiento operativo de la industria.



## Objetivos

### Objetivo General

Desarrollar un sistema computacional integral en C++ para el control de producción industrial, que combine estructuras de datos dinámicas y un motor de inferencia de lógica proposicional con modelos de matemáticas discretas, probabilidad y teoría de orden, permitiendo evaluar, clasificar, jerarquizar y predecir la eficiencia operativa de las líneas de ensamblaje bajo un análisis estricto de complejidad algorítmica.

### Objetivos Específicos

1. **Implementar un motor de inferencia lógica y manejo de conjuntos:** Evaluar el estado de las líneas de producción aplicando lógica proposicional (cumplimiento de metas, fallos mecánicos, validaciones) y organizar los resultados mediante operaciones de conjuntos (unión, intersección y diferencia) sobre estructuras dinámicas.
2. **Gestionar la memoria y el flujo de datos dinámicamente:** Utilizar listas enlazadas simples para el registro principal de las líneas, pilas para el historial de operaciones y colas para priorizar las revisiones, garantizando un control estructurado de la información.
3. **Modelar escenarios mediante combinatoria y probabilidad discreta:** Aplicar principios de multiplicación y combinaciones para determinar el espacio muestral de los estados teóricos de producción, calculando la probabilidad de ocurrencia de operaciones normales, bajo rendimiento y fallos.
4. **Estructurar la jerarquía operativa mediante un orden parcial:** Organizar los distintos niveles de eficiencia de la planta como un conjunto parcialmente ordenado,

representándolo visualmente a través de un Diagrama de Hasse para facilitar la toma de decisiones gerenciales.

5. **Evaluar el rendimiento algorítmico del sistema:** Plantear ecuaciones de recurrencia y realizar un análisis de complejidad computacional de las operaciones y búsquedas implementadas, relacionando el costo del procesamiento con la escalabilidad de la jerarquía productiva.

## Marco Teorico

### 1. Enfoque y Tipo de Proyecto

Para el desarrollo de este proyecto adoptamos un enfoque aplicado y cuantitativo. La investigación es de tipo descriptiva y experimental, ya que buscamos modelar un problema real (el control de producción industrial en "TechFactory") utilizando fundamentos matemáticos y evaluando su comportamiento mediante un simulador en consola. La metodología se centra en traducir reglas de negocio a modelos matemáticos discretos para automatizar la toma de decisiones.

### 2. Diseño del Modelo Lógico (Lógica Proposicional)

El núcleo de la toma de decisiones del sistema se fundamenta en la lógica proposicional. Definimos tres variables proposicionales principales para evaluar el estado diario de cada línea de producción:

- P: La línea cumple o supera la meta de producción diaria.
- Q: La línea no registra fallos mecánicos.
- R: El supervisor valida la operación de la línea.

A partir de estas variables, construimos reglas de inferencia para determinar el estado de la máquina. Establecimos prioridades para evitar ambigüedades, aplicando las siguientes implicaciones lógicas:

- Para que una línea se considere **Operativa**, debe cumplir la conjunción de las tres proposiciones:  $P \wedge Q \wedge R \rightarrow \text{Operativa}$ .
- Si se detectan fallos, el sistema aplica la negación de Q como regla de máxima prioridad:  $\neg Q \rightarrow \text{Detenida}$ .

- Si no hay fallos, pero no se cumple la meta, se aplica:  $\neg P \wedge Q \rightarrow$

Bajo Rendimiento.

### 3. Agrupación y Teoría de Conjuntos

Para administrar el estado general de la planta, empleamos la teoría de conjuntos.

Clasificamos las líneas de producción en tres conjuntos disjuntos basados en el estado inferido previamente: el conjunto  $A$  (líneas operativas), el conjunto  $B$  (bajo rendimiento) y el conjunto  $C$  (detenidas).

Para el análisis de incidencias y la optimización de recursos, programamos operaciones entre conjuntos:

**Intersección ( $A \cap B$ ):** Aunque teóricamente son conjuntos disjuntos en nuestro modelo base, se implementó para prever escenarios donde una línea operativa entre en revisión.

**Diferencia ( $A - C$ ):** Para extraer estrictamente las líneas activas que no presentan fallos graves.

**Unión ( $B \cup C$ ):** Para agrupar todas las líneas que requieren atención técnica o revisión de metas.

### 4. Análisis Combinatorio y Probabilidad Discreta

Implementamos un módulo de análisis teórico para evaluar la escalabilidad y los riesgos del sistema. Utilizando el principio fundamental del conteo, calculamos el espacio muestral de estados posibles multiplicando los niveles de cada variable (Meta, Falla, Validación).

Para la selección de líneas óptimas sin importar el orden, programamos la función de combinaciones basada en la fórmula factorial:

$$C(n, k) = \frac{n!}{k!(n-k)!}$$

Adicionalmente, aplicamos probabilidad discreta clásica (Casos Favorables / Casos Totales) para que el sistema pueda calcular en tiempo de ejecución el riesgo de que una línea teórica falle o presente bajo rendimiento.

### **5. Relaciones de Orden y Complejidad (Ecuaciones de Recurrencia)**

Para establecer una jerarquía de eficiencia entre los diferentes estados de producción, modelamos un conjunto parcialmente ordenado (Poset). Definimos la relación "menor o igual eficiencia" verificando que se cumplan simultáneamente las condiciones de mayor producción, menores fallos y mayor validación. Esta estructura de orden parcial se representa visualmente mediante la generación en consola de un Diagrama de Hasse, mostrando únicamente las relaciones de cobertura.

Finalmente, el costo computacional de evaluar nuevas líneas se modeló mediante una ecuación de recurrencia lineal no homogénea:

$$T(n) = T(n-1) + 777$$

Donde  $T(0) = 0$  Esta ecuación se resuelve en el código demostrando que el sistema tiene un crecimiento  $O(n)$ , asegurando que nuestra implementación escala de manera predecible a medida que crece la fábrica.

### **6. Herramientas y Estructuras de Datos**

El sistema fue desarrollado en C++ puro. Para demostrar un control total sobre el manejo de memoria y aplicar los conceptos teóricos desde cero, restringimos el uso de librerías de alto nivel (como `<string>` o `<iomanip>`). Toda la información se gestiona mediante arreglos de

caracteres, listas enlazadas simples (para el repositorio principal), pilas (para el historial LIFO de operaciones) y colas (para la atención FIFO de líneas detenidas).

## **Diseño y Arquitectura del Sistema**

### **Definición de Estados de Producción (Principios de conteo)**

El sistema establece los estados de producción a partir de tres variables fundamentales que describen el rendimiento de una línea industrial. Dado que cada variable posee un número finito de valores, es posible construir un espacio de estados exhaustivo mediante principios combinatorios.

#### ***Variables de Estado***

Se han seleccionado tres variables para modelar el comportamiento de la línea:

- Meta (M): Representa el nivel de cumplimiento de la producción. Toma valores de 0 para un nivel bajo, 1 para medio y 2 para alto.
- Falla (F): Indica si existen averías mecánicas, con valores 0 (ausencia de fallos) o 1 (presencia de fallos).
- Validación (V): Registra la supervisión humana, donde 0 significa que no ha sido validado y 1 que sí lo ha sido.

Estas variables reflejan dimensiones críticas de la operación: el rendimiento cuantitativo, la disponibilidad operativa y el control de calidad.

#### ***Construcción del Espacio de Estados***

El número total de estados posibles se determina mediante el principio multiplicativo. Al multiplicar las opciones de cada variable ( $3 \times 2 \times 2$ ), se obtienen 12 combinaciones únicas. Cada estado se formaliza como una terna (M, F, V). Los doce estados resultantes cubren todas las posibilidades de la operación, desde una producción baja con fallos y sin validar, hasta una producción alta operativa y supervisada.

### ***Implementacion tecnica***

En el software desarrollado, la funcion listarEstadosTeoricos genera este espacio mediante ciclos anidados. Esta estructura asegura que se recorran todas las combinaciones de las variables de forma sistematica, garantizando que el espacio muestral sea completo y carezca de duplicados.

### ***Relacion con la logica proposicional***

Las variables de estado corresponden a las proposiciones atomicas del sistema de inferencia. La variable de meta se asocia a la comparacion entre la produccion actual y la meta diaria; la falla depende de la ausencia de errores mecanicos; y la validacion se vincula al estado de verificacion del supervisor. Esta equivalencia permite que el analisis teorico de los estados sirva de base directa para la logica de clasificacion empleada en la gestion de las lineas.

### **Análisis de Escenarios (Combinaciones)**

La globalización ha generado una transformación significativa en la forma en que las sociedades interactúan, comparten información y desarrollan sus economías. Esta interconexión ha traído consigo múltiples beneficios, como el acceso a nuevas tecnologías y mercados, pero también ha planteado desafíos importantes en términos de identidad cultural, sostenibilidad y equidad. Comprender estos efectos resulta clave para formular estrategias que permitan un desarrollo más equilibrado y justo.

### **Selección de Casos**

Para comprender el impacto operativo y planificar estrategias de mantenimiento o auditoría, se definen los siguientes casos de estudio donde la combinatoria es clave:



- **Auditoría de Calidad Aleatoria (Líneas Operativas):** De un conjunto  $A$  de líneas que operan sin fallos, el sistema selecciona un subgrupo para una inspección detallada. Aquí, la probabilidad de seleccionar una muestra específica se analiza bajo los principios de experimentos de conteo discreto (Illowsky & Dean, 2013).
- **Intervención de Mantenimiento (Líneas con Bajo Rendimiento):** Del conjunto  $B$ , se debe elegir un número específico de líneas para asignar los equipos de mantenimiento.
- **Análisis de Fallos Simultáneos (Líneas Detenidas):** Evaluación de escenarios críticos donde múltiples líneas del conjunto  $C$  presentan fallos graves, permitiendo al sistema estimar el esfuerzo logístico necesario para la recuperación de la planta.

### *Aplicación de fórmulas combinatorias*

Para calcular el número de escenarios posibles en los casos anteriores, el sistema implementa la fórmula general de combinaciones, la cual determina la cantidad de subconjuntos de tamaño  $k$  a partir de un conjunto mayor  $n$ . Este modelo es fundamental para la gestión eficiente de recursos en sistemas computacionales complejos (Doerr & Levasseur, 2021):

$$C(n, k) = n! / (k! (n - k)!)$$

Donde:

- $n$  es el número total de líneas disponibles en un estado específico.
- $k$  es el número de líneas a seleccionar.

### **Ejemplo aplicado:**

Si el sistema clasifica 10 líneas en el conjunto de Bajo Rendimiento ( $n = 10$ ) y se dispone de recursos para diagnosticar 3 líneas simultáneamente ( $k = 3$ ), el cálculo es:

$$C(10,3) = \frac{10!}{3! (10 - 3)!} = \frac{10 \times 9 \times 8}{3 \times 2 \times 1} = 120$$

Esto demuestra que existen 120 combinaciones distintas de líneas para intervención. Integrar este cálculo permite a TechFactory anticipar la carga de trabajo y optimizar la respuesta operativa, transformando datos brutos en inteligencia de negocios basada en modelos matemáticos comprobables.

### **Modelo de Probabilidad Discreta**

La globalización ha generado una transformación significativa en la forma en que las sociedades interactúan, comparten información y desarrollan sus economías. Esta interconexión ha traído consigo múltiples beneficios, como el acceso a nuevas tecnologías y mercados, pero también ha planteado desafíos importantes en términos de identidad cultural, sostenibilidad y equidad. Comprender estos efectos resulta clave para formular estrategias que permitan un desarrollo más equilibrado y justo.

Definición del espacio muestral

*tral*

- Comprender estos efectos resulta clave
- **para formular estrategias** que permitan un desarrollo más equilibrado y justo.
- Comprender estos efectos resulta cla
- **ve para formular estrategias** que permit
- **an un desarrollo más equilibrado** y justo.
- Comprender estos efectos resulta clave para formular estrategias que permitan un desarrollo más equilibrado y justo.

### ***Cálculo de las probabilidades***

El uso de herramientas digitales en entornos educativos ha demostrado un potencial considerable para mejorar el aprendizaje, especialmente en contextos donde el acceso a recursos tradicionales es limitado. Sin embargo, la brecha digital persiste como una barrera que impide una participación equitativa. Es necesario que las políticas públicas y las instituciones académicas trabajen conjuntamente para garantizar una inclusión real, promoviendo tanto la infraestructura como la capacitación adecuada.

### **Ecuaciones de Recurrencia**

El estudio del costo computacional permite observar cómo se incrementa el tiempo de ejecución al aumentar el número de líneas evaluadas. Modelar este comportamiento sirve para proyectar el desempeño en escalas mayores y confirmar si el enfoque propuesto es realmente factible en términos prácticos.

### ***Identificar Operaciones Críticas***

El proceso central del sistema consiste en evaluar el estado de cada línea de producción mediante la comparación de tres variables específicas meta, falla y validación con los 12 estados definidos en el modelo. Esta tarea constituye la operación base para el cálculo del costo computacional. Al ser fijo el número de estados posibles, el costo de procesar una línea individual no varía. No obstante, el esfuerzo computacional total del sistema escala de forma proporcional al incrementarse el número de líneas, manteniendo una relación de complejidad lineal.

### ***Planteamiento de la Ecuación de Recurrencia***

Sea  $T(n)$  el costo total de inspeccionar  $n$  líneas de producción. El costo de procesar  $n$  líneas se define como la suma del costo de las  $n$  menos 1 líneas anteriores y el costo de evaluar la línea actual. Dado que cada nueva línea requiere 777 comparaciones, correspondientes a cada combinación de evaluación posible, se establece la siguiente relación de recurrencia:

$$T(n) = \begin{cases} 0 & \text{para } n = 0 \\ T(n-1) + 777 & \text{para } n \geq 1 \end{cases}$$

Este planteamiento se refleja directamente en la implementación computacional a través de la función `costoRecurrencia`, la cual calcula este valor de forma recursiva devolviendo 0 cuando  $n$  es igual a 0, o bien la suma de 777 y el resultado de la llamada para  $n$  menos 1 en los demás casos.

Resolviendo la ecuación por sustitución inversa, la forma de la recurrencia implementada en la función `costoFormaCerrada(int n)` es la siguiente:

$$T(n) = 777n$$

### ***Análisis del crecimiento***

La expresión  $T(n) = 777n$  describe un crecimiento lineal en relación con el volumen de líneas analizadas. Este comportamiento indica que el coste computacional evoluciona de manera proporcional a la entrada, de modo que un incremento al doble en el número de líneas resulta en una duplicación del tiempo de ejecución. Esta relación lineal asegura la ausencia de cuellos de botella derivados de una complejidad cuadrática o exponencial. El valor 777 representa el coste asociado al procesamiento de cada línea y sus comparaciones internas. Si bien esta constante puede variar según los detalles de la implementación técnica, su magnitud no modifica la complejidad asintótica del sistema.

## Diagrama de Hasse y Analisis de Complejidad

La globalización ha generado una transformación significativa en la forma en que las sociedades interactúan, comparten información y desarrollan sus economías. Esta interconexión ha traído consigo múltiples beneficios, como el acceso a nuevas tecnologías y mercados, pero también ha planteado desafíos importantes en términos de identidad cultural, sostenibilidad y equidad. Comprender estos efectos resulta clave para formular estrategias que permitan un

## Pruebas

Para validar el funcionamiento del sistema, se llevaron a cabo cuatro pruebas. Cada una de ellas se orientó a un módulo específico del análisis: el conteo de estados, el cálculo de probabilidades, la resolución de la ecuación de recurrencia y la jerarquía de estados representada mediante el diagrama de Hasse.

### Prueba de Conteo

Esta prueba tuvo como objetivo verificar que el número total de estados posibles se determinara correctamente aplicando el principio de multiplicación. Para ello, se configuró el módulo de conteo con los parámetros  $nMeta$  igual a 3,  $nFalla$  igual a 2 y  $nValidado$  igual a 2.

El resultado esperado de 12 estados posibles coincidió con el obtenido por el sistema. El software generó un listado exhaustivo que detalla cada combinación teórica de las variables Meta (Baja, Media, Alta), Falla (No, Si) y Validado (No, Si). Los resultados confirman que el conteo es preciso y contempla todas las permutaciones del modelo.

### **Prueba de Probabilidad**

El propósito de esta fase fue calcular las probabilidades discretas para los eventos de falla, bajo rendimiento y operación normal. Se empleó la fórmula de probabilidad clásica basada en la relación entre casos favorables y totales sobre el conjunto de 12 estados.

Para el evento de falla, se consideró 1 nivel sobre 2 posibles, resultando en una probabilidad de 0.5. El bajo rendimiento, definido por 1 nivel de meta sobre 3, arrojó un valor de 0.3333. Finalmente, la operación normal, que requiere la ausencia de fallas y una validación positiva, resultó en una probabilidad de 0.25. Los valores obtenidos en la ejecución coinciden con los cálculos teóricos, lo que valida la consistencia del modelo probabilístico implementado.

### **Prueba de Recurrencia**

Se evaluó si el costo computacional presenta un crecimiento lineal respecto al número de líneas, comparando la ejecución de la función recursiva  $T(n) = T(n-1) + 777$  con su forma cerrada  $T(n) = 777n$ .

Al ejecutar el módulo para valores de  $n$  del 1 al 5, se observó que ambas expresiones producen resultados idénticos en todos los puntos de control. Esta correspondencia confirma que la recurrencia fue resuelta correctamente. Asimismo, la complejidad lineal  $O(n)$  asegura que el sistema mantiene su eficiencia operativa conforme aumenta el volumen de datos.

### **Prueba de Jerarquía**

El objetivo de este módulo fue representar el orden parcial de eficiencia entre los estados. Se estableció que un estado A es menor o igual a B si el segundo es igual o superior en las tres variables: mayor nivel de meta, menor presencia de fallas y validación positiva.

El sistema generó una matriz de relación de 12 por 12 donde los valores binarios indican la existencia de un vínculo de orden. La diagonal principal compuesta por unos confirma la propiedad reflexiva. Los valores fuera de la diagonal muestran la superioridad de eficiencia de un estado sobre otro, mientras que los ceros señalan estados incomparables donde no se cumple la mejora simultánea en todas las variables. La matriz de la relación se muestra en la *Tabla1* la cual es complementado por la *Tabla2*.

Complementariamente, el diagrama de Hasse permite visualizar las relaciones de cobertura, omitiendo los estados intermedios para simplificar la estructura. Esta doble representación, matricial y gráfica, describe de forma coherente la jerarquía del sistema y los tránsitos directos entre los distintos niveles de eficiencia.

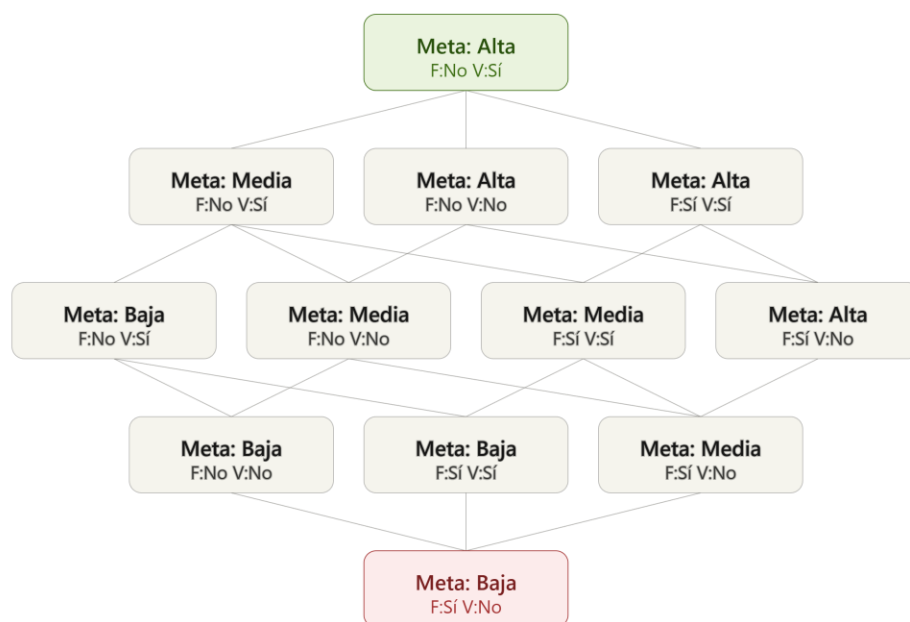
## **Complejidad**

El proyecto combina algoritmos de costo constante (conteo, probabilidad) con otros lineales (factorial, recurrencia de costo  $T(n)=777n$ ) y uno cúbico (la construcción del diagrama de Hasse,  $O(E^3)$ ). Como el modelo teórico usa  $E=12$  estados fijos, ese costo cúbico es despreciable en la práctica, pero crecería rápido si se ampliara el número de variables del modelo. En conjunto, el sistema escala de forma predecible: ninguna operación depende de un crecimiento exponencial. El resumen de complejidad se encuentra en la Tabla 3.

## **Figuras**

### **Figura 1**

*1 Diagrama de Hasse del Algoritmo*



*Nota. Rojo: mínimo del orden (peor estado) · Verde: máximo (mejor estado) · una línea sube de A hacia B si B cubre a A*



## Tablas

**Tabla 1**

*Matriz Para Representar El Grafo De La Relacion De Las Líneas.*

| Estado | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|--------|---|---|---|---|---|---|---|---|---|----|----|----|
| 1      | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1  | 0  | 0  |
| 2      | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1  | 0  | 0  |
| 3      | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0  | 1  | 1  |
| 4      | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0  | 0  | 1  |
| 5      | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1  | 0  | 0  |
| 6      | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1  | 0  | 0  |
| 7      | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0  | 1  | 1  |
| 8      | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0  | 0  | 1  |
| 9      | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1  | 0  | 0  |
| 10     | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1  | 0  | 0  |
| 11     | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0  | 1  | 1  |
| 12     | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0  | 0  | 1  |

*Nota. Matriz Para Representar El Grafo De La Relacion De Las Líneas.*

**Tabla 2**

*Significado de cada estado de la matriz para representar el grafo de la relacion de las líneas.*

*Tabla 3*

| # | Meta  | Falla | Validado | Etiqueta       |
|---|-------|-------|----------|----------------|
| 1 | Baja  | No    | No       | (B,N,N)(B,N,N) |
| 2 | Baja  | No    | Si       | (B,N,S)(B,N,S) |
| 3 | Baja  | Si    | No       | (B,S,N)(B,S,N) |
| 4 | Baja  | Si    | Si       | (B,S,S)(B,S,S) |
| 5 | Media | No    | No       | (M,N,N)(M,N,N) |
| 6 | Media | No    | Si       | (M,N,S)(M,N,S) |

|           |       |    |    |                |
|-----------|-------|----|----|----------------|
| <b>7</b>  | Media | Si | No | (M,S,N)(M,S,N) |
| <b>8</b>  | Media | Si | Si | (M,S,S)(M,S,S) |
| <b>9</b>  | Alta  | No | No | (A,N,N)(A,N,N) |
| <b>10</b> | Alta  | No | Si | (A,N,S)(A,N,S) |
| <b>11</b> | Alta  | Si | No | (A,S,N)(A,S,N) |
| <b>12</b> | Alta  | Si | Si | (A,S,S)(A,S,S) |

*Nota. Significado De Cada Estado De La Matriz Para Representar El Grafo De La Relacion De Las Líneas.*

### Tabla 3

*Complejidad de las Funciones en el Algoritmo*

*Tabla 4*

| <b>Función</b>               | <b>Qué hace</b>                                                   | <b>Complejidad</b>         | <b>Justificación</b>                                                                                                  |
|------------------------------|-------------------------------------------------------------------|----------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>contarEstadosPosibles</b> | Multiplica<br>$n_{Meta} \times n_{Falla} \times n_{Va}$<br>lidado | $O(1)$                     | Una sola multiplicación.                                                                                              |
| <b>listarEstadosTeoricos</b> | Triple for anidado<br>con límites fijos<br>(3,2,2)                | $O(E)$<br>conceptual       | Como los límites están<br>escritos como literales (no<br>como parámetro), el<br>compilador siempre itera<br>12 veces. |
| <b>factorial (recursiva)</b> | Calcula $n!$                                                      | $O(n)$ tiempo<br>y espacio | $n$ llamadas recursivas,<br>cada una $O(1)$ ; la pila de<br>llamadas crece<br>linealmente.                            |

|                                          |                                                                                                |                                   |                                                                                                  |
|------------------------------------------|------------------------------------------------------------------------------------------------|-----------------------------------|--------------------------------------------------------------------------------------------------|
| <b>combinaciones(n,k)</b>                | Llama a factorial 3 veces: n, k, n-k                                                           | O(n)                              | Como $k \leq n$ , el término dominante es O(n). Ojo: long long se desborda para $n \gtrsim 20$ . |
| <b>probabilidadCondicion y derivadas</b> | Una división de dos productos                                                                  | O(1)                              | Aritmética directa, sin bucles.                                                                  |
| <b>costoRecurrencia(n)</b>               | $T(n) = T(n-1) + 777$ , recursiva                                                              | O(n) tiempo, O(n) espacio de pila | n llamadas anidadas.                                                                             |
| <b>costoFormaCerrada(n)</b>              | $777 \cdot n$ directo                                                                          | O(1)                              | Forma cerrada de la recurrencia anterior.                                                        |
| <b>construirDiagramaHasse</b>            | Por cada par (i,j) de E estados revisa si hay un intermedio recorriendo otra vez los E estados | O(E <sup>3</sup> )                | Doble bucle O(E <sup>2</sup> ) × verificación de intermedio O(E) en existeIntermedio.            |
| <b>analisisComplejidad</b>               | Solo imprime texto                                                                             | O(1)                              | Sin bucles dependientes de datos.                                                                |

*Nota.* La tabla muestra el análisis de complejidad de cada función en el algoritmo y describe lo que hace..

## Conclusiones

**Respecto a la automatización lógica (Objetivo 1):** Se logró diseñar un motor de inferencia capaz de clasificar el estado de las líneas en tiempo real. La aplicación de la lógica proposicional permitió que el sistema actúe como un supervisor virtual, mientras que la teoría de conjuntos facilitó la gestión de grupos de líneas según su rendimiento operativo.

**Respecto a la gestión de datos (Objetivo 2):** Se confirmó que la implementación de estructuras dinámicas (listas, pilas y colas) optimiza el manejo de la información. El uso de pilas resultó efectivo para el historial operativo, y las colas permitieron una gestión jerárquica de las líneas que requieren revisión inmediata, garantizando un flujo de trabajo ordenado.

**Respecto al análisis probabilístico (Objetivo 3):** La integración de principios de conteo y combinatoria transformó el sistema en una herramienta de análisis predictivo. Fue posible dimensionar matemáticamente el espacio de posibles escenarios de producción, lo que permite a la empresa prepararse ante contingencias de fallos o bajo rendimiento antes de que impacten en la planta.

**Respecto a la jerarquía operativa (Objetivo 4):** A través de la teoría de órdenes, se construyó el Diagrama de Hasse, logrando representar visualmente la eficiencia. Esta estructura permitió organizar los estados operativos de manera lógica, facilitando la identificación de las relaciones de dependencia entre los niveles de rendimiento de cada línea.

**Respecto al rendimiento computacional (Objetivo 5):** Finalmente, el análisis de complejidad y las ecuaciones de recurrencia validaron que nuestro software es eficiente.

Se determinó que, gracias a la implementación de estructuras dinámicas, el sistema es escalable y capaz de mantener un costo de procesamiento óptimo incluso cuando el número de líneas evaluadas aumenta, asegurando la viabilidad técnica del proyecto a largo plazo.

### Referencias Bibliográficas

Castells, M. (2010). *The rise of the network society* (2nd ed.). Wiley-Blackwell.

González, R., & Martínez, L. (2021). Educación virtual y equidad digital en América Latina.

*Revista Latinoamericana de Educación*, 55(2), 45–62.

<https://doi.org/10.1234/rle.2021.55.2.45>

Organización Mundial de la Salud. (2023, marzo 15). *Salud mental: Fortalecimiento de la*

*respuesta efectiva en tiempos de crisis*. [https://www.who.int/es/news-room/detail/15-03-](https://www.who.int/es/news-room/detail/15-03-2023-salud-mental-crisis)

2023-salud-mental-crisis

En las referencias bibliográficas se debe aplicar sangría francesa y deben ir organizadas alfabéticamente.

Doerr, A., & Levasseur, K. (2021). *Applied Discrete Structures*. Matemáticas Discretas

Aplicadas a la Computación.

<https://discretemath.org>

